Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Vapor Stone Rail Systems

Generic Software Documentation Project

Project # 7260

Software Design Description (SDD)
Proportional Integral Derivative (PID) Module

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	James Kim Software Designer	James Kim 2011-09-12
Verified :	Eric Malo, Eng. System Engineer	2011-09-12
Approved :	Martin Ouellet, Eng. Software Department Supervisor	2011-09-12

Document No.	**201111554**	Revision	**0**
		Date (yyyy-mm-dd)	**2011-08-17**

Revision List

Rev.	ECD#	Date	By	Description of changes
0	---	2011-08-17	James Kim	Original issue

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998 and 1558-2004.

Table of Contents

Revision List 2
Preface 3
Table of Contents 4
List of Figures 5
List of Tables 5

1 INTRODUCTION 6

1.1 Purpose 6
1.2 Scope 6
1.3 Definitions and Acronyms 6
1.3.1 Definitions 6
1.3.2 Acronyms 6
1.4 Document Organization 7

2 REFERENCES 8

3 SOFTWARE DESIGN 9

3.1 Decomposition Description 9
3.1.1 Module Decomposition 9
3.1.2 Concurrent Process Decomposition 9
3.1.3 Data Decomposition 10
3.2 Dependency Description 12
3.2.1 Inter-Module Dependencies 12
3.2.2 Inter-Process Dependencies 12
3.2.3 Data Dependencies 12
3.3 Interface Description 13
3.3.1 Module Interfaces 13
3.4 Detailed Design 17
3.4.1 PID Module Detailed Design 17
3.4.2 Data Entities Detailed Design 34

List of Figures

Figure 3-1: Inter-Module Dependencies Diagram 12
Figure 3-2: PID_Init Flowchart 17
Figure 3-3: PID_LFIL Flowchart 17
Figure 3-4: PID_LTRJ Flowchart 18
Figure 3-5: PID_LPES Flowchart 19
Figure 3-6: PID_LPEI Flowchart 20
Figure 3-7: PID_LPEB Flowchart 20
Figure 3-8: PID_DFH Flowchart 20
Figure 3-9: PID_RDRP Flowchart 21
Figure 3-10: PID_RDDP Flowchart 21
Figure 3-11: PID_RDRV Flowchart 21
Figure 3-12: PID_RDDV Flowchart 22
Figure 3-13: PID_RDSUM Flowchart 22
Figure 3-14: PID_RSTI Flowchart 22
Figure 3-15: PID_STT Flowchart 23
Figure 3-16: PID_STT_Cmd Flowchart 24
Figure 3-17: pid_Velocity_Move_State Flowchart 27
Figure 3-18: pid_PWM_Move_State Flowchart 28
Figure 3-19: pid_Profile_Move_State Flowchart 29
Figure 3-20: pid_Stop_Smooth_State Flowchart 30
Figure 3-21: pid_Stop_Abrupt_State Flowchart 31
Figure 3-22: PID_CommandTask Flowchart 33

List of Tables

Table 3-1: Structure of PID Filter Load 10
Table 3-2: Structure of PID Trajectory Load 10
Table 3-3: Structure of Profile Generation Segment Storage (MoveCommandStruct) 11
Table 3-4: Value Specific to the Project 19
Table 3-5: Calculation of the Door Operator Parameters 19
Table 3-6: Equation for Final Accel, Decel and Speed Calculation 26

1 INTRODUCTION

This document is the Software Design Description of the Proportional Integral Derivative (PID) module developed by Vapor Stone Rail Systems.

The PID module is a part of the Door Control Module (DCM) Application that manages the motor control feature.

1.1 Purpose

The purpose of this Software Design Description (SDD) is to describe the software components and implementation required details of the PID module.

The intended audience is the Vapor Stone Rail Systems software development team and maintenance team members of the transit authority and the car builder.

1.2 Scope

The PID module is a software module used for controlling a motor of a door operator; therefore it is safety related module.
This module interfaces with other modules and, it is responsible for a door operation.

1.3 Definitions and Acronyms

1.3.1 Definitions

Door Control Module:	A specific processor based electronic module running a Door Control Module Application. It is used for controlling the door motor.

1.3.2 Acronyms

Acronym	Meaning
ASCII	American Standard Code for Information Interchange
CDRL	Contract Data Requirements List
CPU	Central Processing Unit
DFH	Define Home
DMS	Door Module System
DOP	Door Operator
ECD	Engineering Change Document
IEEE	Institute of Electrical and Electronics Engineers
Il	Integral Limit
KB	Kilobytes
Kd	Derivative Gain
Ki	Integral Gain
Kp	Proportional Gain
LED	Light Emitting Diode
LFIL	Load Filter
LLC	Limited Liability Corporation
LPEB	Load Position Error for Braking
LPEI	Load Position Error for Interrupt
LPES	Load Position Error for Stopping
LTRJ	Load Trajectory
MHz	MegaHertz
ms	millisecond
NVRAM	Non-Volatile Random Access Memory
OS	Operating System

Acronym	Meaning
PID	Proportional Integral Derivative
PT	Portable Test
PWM	Pulse Width Modulation
RAM	Random Access Memory
RDDP	Read Desired Position
RDDV	Read Desired Velocity
RDRP	Read Real Position
RDRV	Read Real Velocity
RDSUM	Read Integral Sum Error
RSTI	Reset Interrupt
SCI	Software Configuration Items
SDD	Software Design Description
Si	Sampling Interval
SRAM	Static Random Access Memory
SRS	Software Requirements Specification
Std	Standard
STT	Send the Trajectory
TS	Technical Specification
UDF	Update Filter
VSRS	Vapor Stone Rail Systems

1.4 Document Organization

This SDD is divided into three sections and the following specific information is provided within each one:

- Section 1, Introduction, states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, References, lists the factors that affect the product and its requirements.
- Section 3, Software Design, summarizes followings:
    - Decomposition Description: describes functions of software entities.
    - Dependency Description: gives a brief description of the dependency between each entity.
    - Interface Description: describes the interaction between entities.
    - Detailed Design: details the functions implemented by each software entity, presents and explains tasks performed by each software entity in functional conditions.

Within the document, diagrams, flowcharts and tables are used to present the software design:

- Decision flowcharts give a graphical view of how entities and functions process work.
- The diagram Figure 3-1 describes the dependency between different modules.
- Tables contain data structures used by the PID Module. They also present C code.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, September 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2004	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

S17-07	Software Design Description Template, Rev. 0, 2002-01-15[1]

[1] Complemented with IEEE STD 1558 Requirements

3 SOFTWARE DESIGN

3.1 Decomposition Description

3.1.1 Module Decomposition

3.1.1.1 PID Module Description

The PID module is a single entity module performing following functions:

- Calculate the speed profile
- Injecting PWM voltage to H-bridge circuit
- Saving the last door movement information
- Detect the obstruction by calculating the door distance error
- Testing the motor encoder

3.1.2 Concurrent Process Decomposition

3.1.2.1 PID Task Description

The PID task loop runs every 5ms and calls the internal functions. This task is running as an independent thread. It has the highest task priority among other modules.

3.1.3 Data Decomposition

3.1.3.1 PID Data Description

The PID Module uses the following structure to calculate the PID filter.

Table 3-1: Structure of PID Filter Load

Identification	PID_LFIL
Entity Type	Structure PID_LFIL_Struct

Prototype	typedef struct PID_LFIL_Struct
	{
	union _Control_LFIL
	{
	unsigned int uiControl; // Control data word received
	struct _CtrlApart
	{
	unsigned char bLoadIl : 1; // Bit 0 New Il value have been received
	unsigned char bLoadKd : 1; // Bit 1 New Kd value have been received
	unsigned char bLoadKi : 1; // Bit 2 New Ki value have been received
	unsigned char bLoadKp : 1; // Bit 3 New Kp value have been received
	unsigned char ucUnused : 4; // Bit 4..7
	unsigned char ucDerivativeSI: 8; // Bit 8..15, SI for the derivative part
	} CtrlApart;
	} Control;
	
	int iKp; // 16bits value. only 15 bits are used for Kp
	int iKi; // 16bits value. only 15 bits are used for Ki
	int iKd; // 16bits value. only 15 bits are used for Kd
	int iIl; // 16bits value. only 15 bits are used for Il
	
	}PID_LFILStruct;

The PID Module uses the following structure to calculate the PID trajectory profile.

Table 3-2: Structure of PID Trajectory Load

Identification	PID_LTRJ
Entity Type	Structure PID_LTRJ_Received_Struct

Prototype	typedef struct PID_LTRJ_Received_Struct
	{
	union _Control_LTRJ_Rcv
	{
	unsigned int uiControl; // Control data word received
	struct _CtlApart
	{
	unsigned char bRelativePos : 1; // Bit 0 Position data is relative
	unsigned char bLoadPosition : 1; // Bit 1 Position will be loaded
	unsigned char bRelativeVel : 1; // Bit 2 Velocity data is relative
	unsigned char bLoadVelocity : 1; // Bit 3 Velocity will be loaded
	unsigned char bRelativeAccel : 1; // Bit 4 Acceleration data is relative
	unsigned char bLoadAccel : 1; // Bit 5 Acceleration will be loaded
	unsigned char bRelativeDecel : 1; // Bit 6 Deceleration data is relative
	unsigned char bLoadDecel : 1; // Bit 7 Deceleration will be loaded
	unsigned char bMotorOff : 1; // Bit 8 Turn Off Motor (Output zero drive)
	unsigned char bStopAbrupt : 1; // Bit 9 Stop Abruptly (Maximum Deceleration)
	unsigned char bStopSmooth : 1; // Bit 10 Stop Smoothly, decelerate as programmed
	unsigned char bVelocityMode : 1; // Bit 11 Velocity mode
	unsigned char bForwardDir : 1; // Bit 12 Forward Direction (positive) in velocity only
	unsigned char bPWMMode : 1; // Bit 13 PWM mode, dVelocity is the PWM.

```
            unsigned char ucUnused        : 2; // Bit 15 and 14 unused
        } CtlApart;
    } Control;

    float fDecel;                // Follow the signed structure, 14 bits integer and 8 bits fraction
    float fAccel;                // Follow the signed structure, 14 bits integer and 8 bits fraction
    float fVelocity;             // Follow the signed structure, 14 bits integer and 8 bits fraction
    float fPosition;             // Follow the signed structure, 26 bits integer and 8 bits fraction

}PID_LTRJ_ReceivedDataStruct;
```

The PID Module uses the following structure to calculate the speed profile.

Table 3-3: Structure of Profile Generation Segment Storage (MoveCommandStruct)

Identification	Move_1
Entity Type	Structure MoveCommandStruct
Prototype	typedef struct _MoveCommandStruct { unsigned int uiStepExec; struct { double dAccel; unsigned long ulSample; }step[4]; }MoveCommandStruct;

3.2 Dependency Description

3.2.1 Inter-Module Dependencies

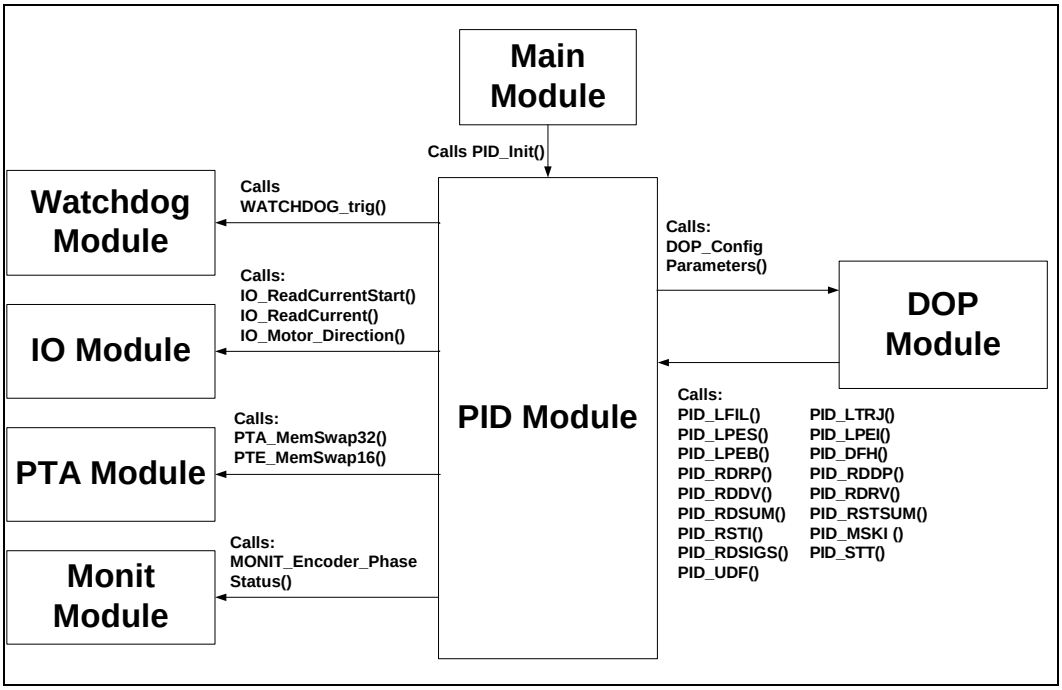

Figure 3-1: Inter-Module Dependencies Diagram

3.2.1.1 PID Module

Figure 3-1 shows the Inter-Module Dependencies Diagram of the PID module.

3.2.2 Inter-Process Dependencies

Not applicable.

3.2.3 Data Dependencies

Not applicable.

3.3 Interface Description

This section describes the functions of PID module, listed in alphabetical order.

3.3.1 Module Interfaces

3.3.1.1 PID Module External Interfaces

Function Name	void PID_CommandTask (void)
Description	Controls the PWM operation and receives command and parameters
Input	None.
Output	None.
Return	None.

Function Name	void PID_DFH (void)
Description	Defines the Home Position of the PID Module. This resets the position encoder count.
Input	None.
Output	None.
Return	None.

Function Name	void PID_Init (void)
Description	Resets the PID module and initializes Encoder inputs, PWM output.
Input	None.
Output	None.
Return	None.

Function Name	void PID_LFIL (PID_LFILStruct *pLFIL_NewData)
Description	Loads the filter parameter into the PID Module.
Input	*pLFIL_NewData: Control word of the LFIL command and PID parameters
Output	None.
Return	None.

Function Name	void PID_LPEB (unsigned int uiPositionError)
Description	Loads the position error on the trajectory of the door into the PID Module, shunts the motor if the trajectory is not followed and generates an interrupt
Input	uiPositionError: Position error value (2 bytes)
Output	None.
Return	None.

Function Name	void PID_LPEI (unsigned int uiPositionError)
Description	Loads the position error on the trajectory of the door into the PID Module, stops the motor if the trajectory is not followed and generates an interrupt
Input	uiPositionError: Position error value (2 bytes)
Output	None.
Return	None.

Function Name	void PID_LPES (unsigned int uiPositionError)
Description	Loads the position error on the trajectory of the door into the PID Module and stop the motor if the trajectory is not followed.
Input	uiPositionError: Position error value (2 bytes)
Output	None.
Return	None.

Function Name	void PID_LTRJ (PID_LTRJ_ReceivedDataStruct *pLTRJ_NewData)
Description	Loads the trajectory of the door into the PID Module.
Input	*pLTRJ_NewData: Control word of the LTRJ command and trajectory value to load
Output	None.
Return	None.

Function Name	void PID_RDDP (long *plPosition)
Description	Reads the desired (theoretical) position of the encoder via the PID Module.
Input	None.
Output	*plPosition: Pointer to the value that will store the desired position.
Return	None.

Function Name	void PID_RDDV (int *piDesiredVelocity)
Description	Reads the desired velocity of the door via the PID Module.
Input	None.
Output	*piDesiredVelocity: Pointer to the value that will store the desired velocity.
Return	None.

Function Name	void PID_RDRP (long *plPosition)
Description	Reads the real position of the encoder via the PID Module.
Input	None.
Output	*plPosition: Pointer to the value that will store the real position.
Return	None.

Function Name	void PID_RDRV (int *piRealVelocity)
Description	Reads the real velocity of the door via the PID Module.
Input	None.
Output	*piRealVelocity: Pointer to the value that will store the desired velocity.
Return	None.

Function Name	void PID_RDSUM (int *piRsum)
Description	Reads the integral sum error of the PID via the PID Module.
Input	None.
Output	*piRsum: Pointer to the value that will store the integral sum error.
Return	None.

Function Name	void PID_RSTI (void)
Description	Reset interrupts. When one of the interrupts of the source is set, this function can reset one or all of the 6 interrupt bits.
Input	None.
Output	None.
Return	None.

Function Name	int PID_STT (void)
Description	Sends a command to the PID Module to start the motor.
Input	None.
Output	None.
Return	Code Return: PID_NO_ERROR = No error occurred PID_STT_ERROR = Command error when STT was sent

Function Name	void PID_UDF (void)
Description	Sends command to the PID Module to use the filters previously loaded with PID_LFIL command and uses them as current parameters.
Input	None.
Output	None.
Return	None.

3.3.1.2 PID Module Internal Interface

Function Name	void pid_Encoder_Test(void)
Description	This function verifies the health of the encoder by identifying the number of pulses per index.
Input	None.
Output	None.
Return	None.

Function Name	void pid_Idle_State (void)
Description	This function is the “do-nothing” state that stops the PWM, and it waits until it goes into another valid state.
Input	None.
Output	None.
Return	None.

Function Name	void pid_Pid_Calculation(void)
Description	This function calculates the new PWM with the current errors
Input	None.
Output	None.
Return	None.

Function Name	void pid_Profile_Move_State (void)
Description	This function executes the PID to obtain the profile move as preprogrammed according to the acceleration, maximum speed and deceleration to reaches the target point programmed for Open Direction.
Input	None.
Output	None.
Return	None.

Function Name	void pid_PWM_Move_State (void)
Description	This function executes the PID to obtain the PWM pre-programmed.
Input	None.
Output	None.
Return	None.

Function Name	void pid_Setup_Move(void)
Description	This function sets up the motor movement profile. It calculates the first deceleration in case the initial speed is higher (or lower if negative) than the maximum speed. It also monitors the movement if it is in trapezoidal shaped profile.
Input	None.
Output	None.
Return	None.

Function Name	void pid_Stop_Abrupt_State (void)
Description	This function stops the motor at maximum torque as possible by fixing the next point to current at time STT command is executed.
Input	None.
Output	None.
Return	None.

Function Name	void pid_Stop_Smooth_State (void)
Description	This function takes care of stopping the motor at the pre-programmed deceleration rate
Input	None.
Output	None.
Return	None.

Function Name	void pid_Velocity_Move_State (void)
Description	This function executes the PID to obtain the velocity pre-programmed.
Input	None.
Output	None.
Return	None.

3.4 Detailed Design

This section details all operations performed by PID Module.

3.4.1 PID Module Detailed Design

The PID module is in charge of motor control. This module uses encoder phase A and B inputs as feedback for the motor position in order to control the PWM and the direction of the motor. The following sub-sections describe each function.

3.4.1.1 PID_Init Function

This function puts the PID module in the desired state to be used by the program. The Main Module calls this function. Figure 3-2 shows the processing done when the PID_Init function is called.

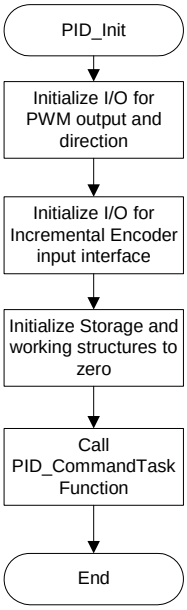

Figure 3-2: PID_Init Flowchart

3.4.1.2 PID_LFIL Function

This function loads the PID filter values. The Door Operator module calls this function. Figure 3-3 shows the processing done when the PID_LFIL function is called.

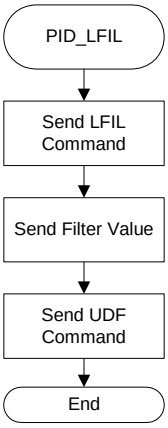

Figure 3-3: PID_LFIL Flowchart

3.4.1.3 PID_LTRJ Function

This function loads the PID trajectory values. The Door Operator module calls this function. Figure 3-4 shows the processing done when the PID_LTRJ function is called.

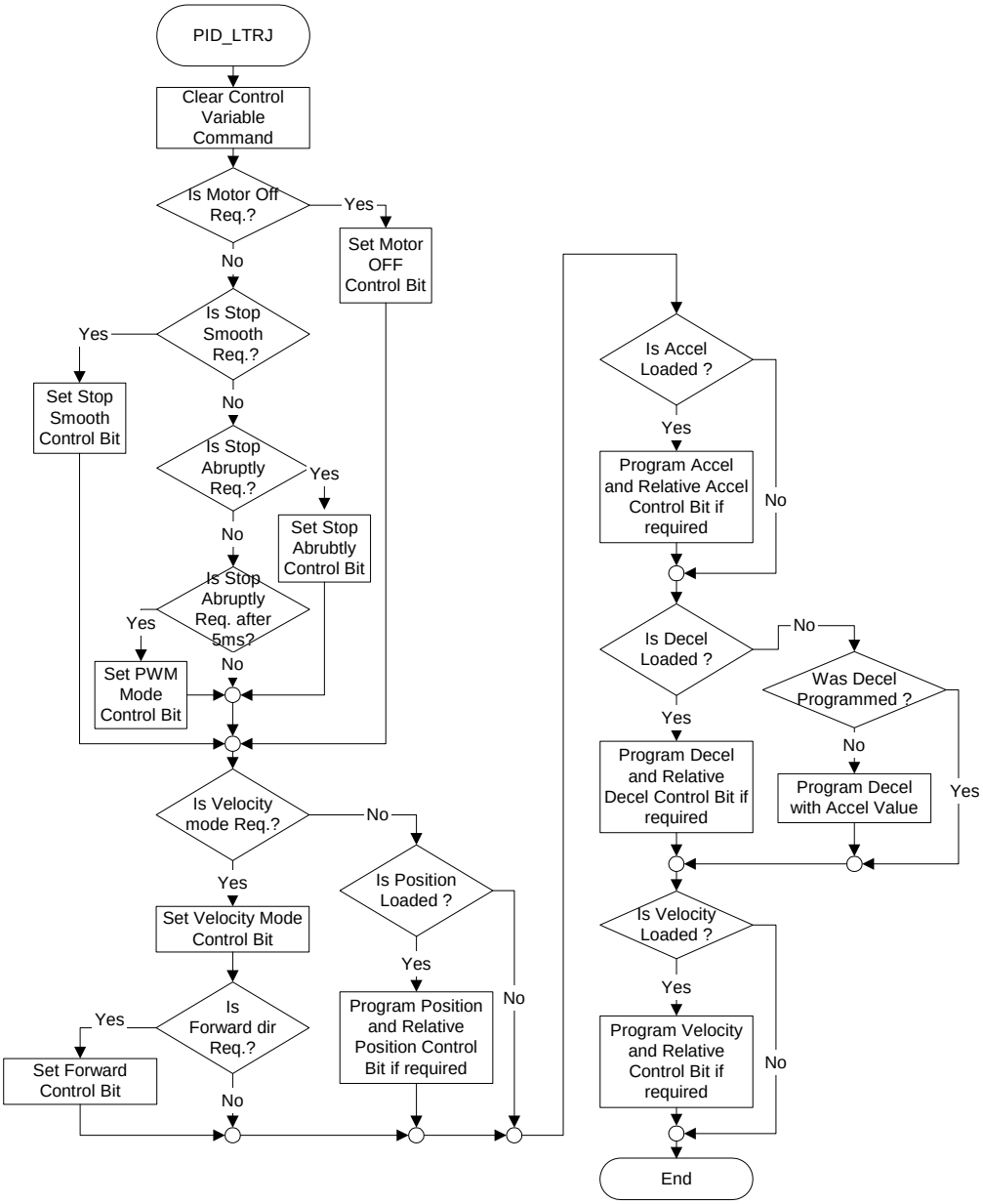

Figure 3-4: PID_LTRJ Flowchart

Input Parameter Calculation:
The PID trajectory parameters are calculated with following equations in Table 3-5. To calculate the parameters, the following specific values must be known before:

Table 3-4: Value Specific to the Project

Name	Symbol	Description
Max door opening	d_{max}	In inch.
Linear distance per revolution of the screw	$ldprev_{screw}$	In inch.
Gear box ratio	r_{gear}	The number of motor rotation per one rotation of the screw.
Encoder resolution	enc_{ppr}	The number of pulses per motor rotation.
Control circuit resolution	res_{factor}	Times factor (ex: X4)
Max speed of door	$v_{maxdoor}$	The speed in inches/second.
Sample period	p_{sample}	The PID Module task scheduling. This is usually 5ms.

Table 3-5: Calculation of the Door Operator Parameters

Parameter	Value
Total number of motor revolution (rev_{total})	d_{max} x r_{gear} / $ldprev_{screw}$ =(Revolution counts)
Total opening counts ($count_{total}$)	enc_{ppr} x r_{gear} x rev_{total} =(Encoder counts)
Linear distance per count ($ldpcnt$)	d_{max} / $count_{total}$ =(Inch per count)
Max speed of motor ($v_{maxmotor}$)	$v_{maxscrew}$ x r_{gear} =(Revolution per second)
Max speed of screw ($v_{maxscrew}$)	($v_{maxdoor}$ / $ldprev_{screw}$) x 60 (sec/min) =(rpm)
Max speed of door in pulse per sample	$v_{maxdoor}$ x r_{gear} x enc_{ppr} x res_{factor} x p_{sample} / $ldprev_{screw}$ =(Pulses/sample)

3.4.1.4 PID_LPES Function

This function loads the PID position error value. The Door Operator module calls this function. Figure 3-5 shows the processing done when the PID_LPES function is called. This command programs the PID module to generate an interrupt upon excessive error and stop the profile in progress.

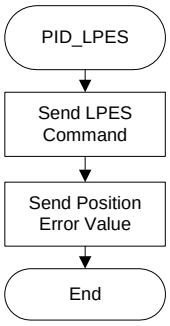

Figure 3-5: PID_LPES Flowchart

3.4.1.5 PID_LPEI Function

This function loads the PID position error value. The Door Operator module calls this function. Figure 3-6 shows the processing done when the PID_LPEI function is called. This command programs the PID module to generate an interrupt upon excessive error but not stopping the profile in progress.

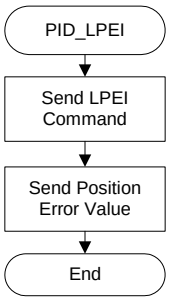

Figure 3-6: PID_LPEI Flowchart

3.4.1.6 PID_LPEB Function

This function loads the PID position error value. The Door Operator module calls this function. Figure 3-5 shows the processing done when the PID_LPEB function is called. This command programs the PID module to generate an interrupt upon excessive error, stop the profile in progress and brake (shunt) the motor.

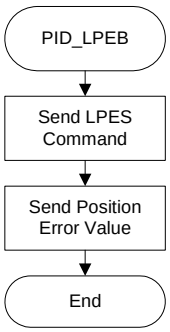

Figure 3-7: PID_LPEB Flowchart

3.4.1.7 PID_DFH Function

This function defines the home position of the PID Module. The Door Operator module calls this function. Figure 3-8 shows the processing done when the PID_DFH function is called.

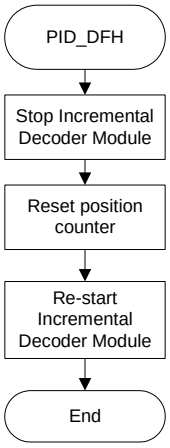

Figure 3-8: PID_DFH Flowchart

3.4.1.8 PID_RDRP Function

This function reads the real position value of the PID Module encoder. The Door Operator module calls this function. Figure 3-9 shows the processing done when the PID_RDRP function is called.

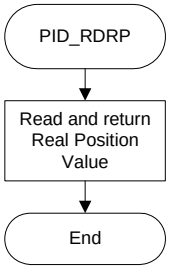

Figure 3-9: PID_RDRP Flowchart

3.4.1.9 PID_RDDP Function

This function reads the desired position value of the PID Module. The Door Operator module calls this function. Figure 3-10 shows the processing done when the PID_RDDP function is called.

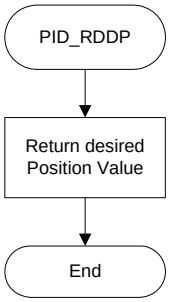

Figure 3-10: PID_RDDP Flowchart

3.4.1.10 PID_RDRV Function

This function reads the real velocity value of the motor. The Door Operator module calls this function. Figure 3-11 shows the processing done when the PID_RDRV function is called.

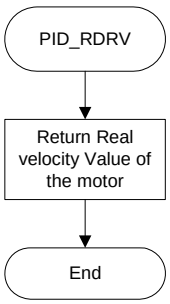

Figure 3-11: PID_RDRV Flowchart

3.4.1.11 PID_RDDV Function

This function reads the desired velocity value of the PID Module. The Door Operator module calls this function. Figure 3-12 shows the processing done when the PID_RDDV function is called.

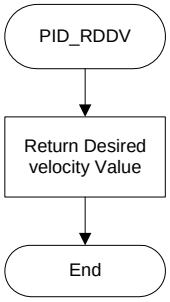

Figure 3-12: PID_RDDV Flowchart

3.4.1.12 PID_RDSUM Function

This function reads the integral sum of PID feedback loop. The Door Operator module calls this function. Figure 3-13 shows the processing done when the PID_RDSUM function is called.

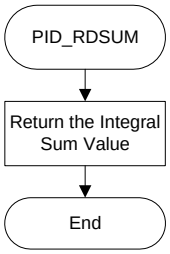

Figure 3-13: PID_RDSUM Flowchart

3.4.1.13 PID_RSTI Function

This function resets the interrupt of the PID module. The Door Operator module calls this function. Figure 3-14 shows the processing done when the PID_RSTI function is called.

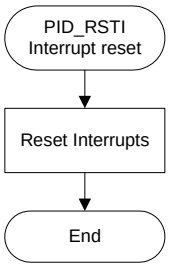

Figure 3-14: PID_RSTI Flowchart

3.4.1.14 PID_STT Function

This function starts the PID Module with the current programmed trajectory and the current set of filters. This function is called by Door Operator module. Figure 3-15 shows the processing done when the PID_STT function is called.

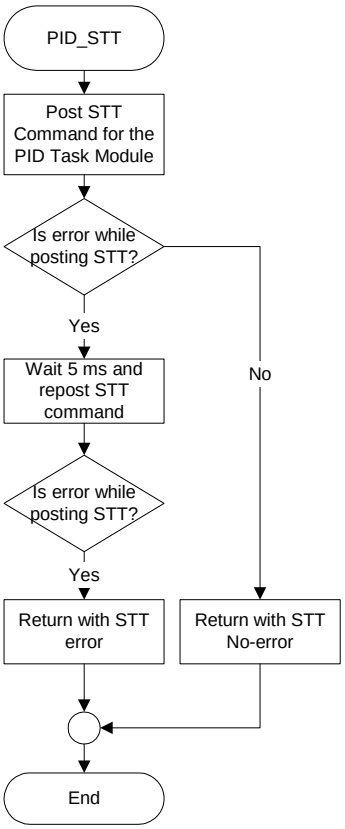

Figure 3-15: PID_STT Flowchart

3.4.1.15 PID_STT_Cmd Function

This function is to parse the STT command whenever a STT command has been posted. This function is called by the PID task when the semaphore has been accepted. Figure 3-16 shows the processing done when function is called.

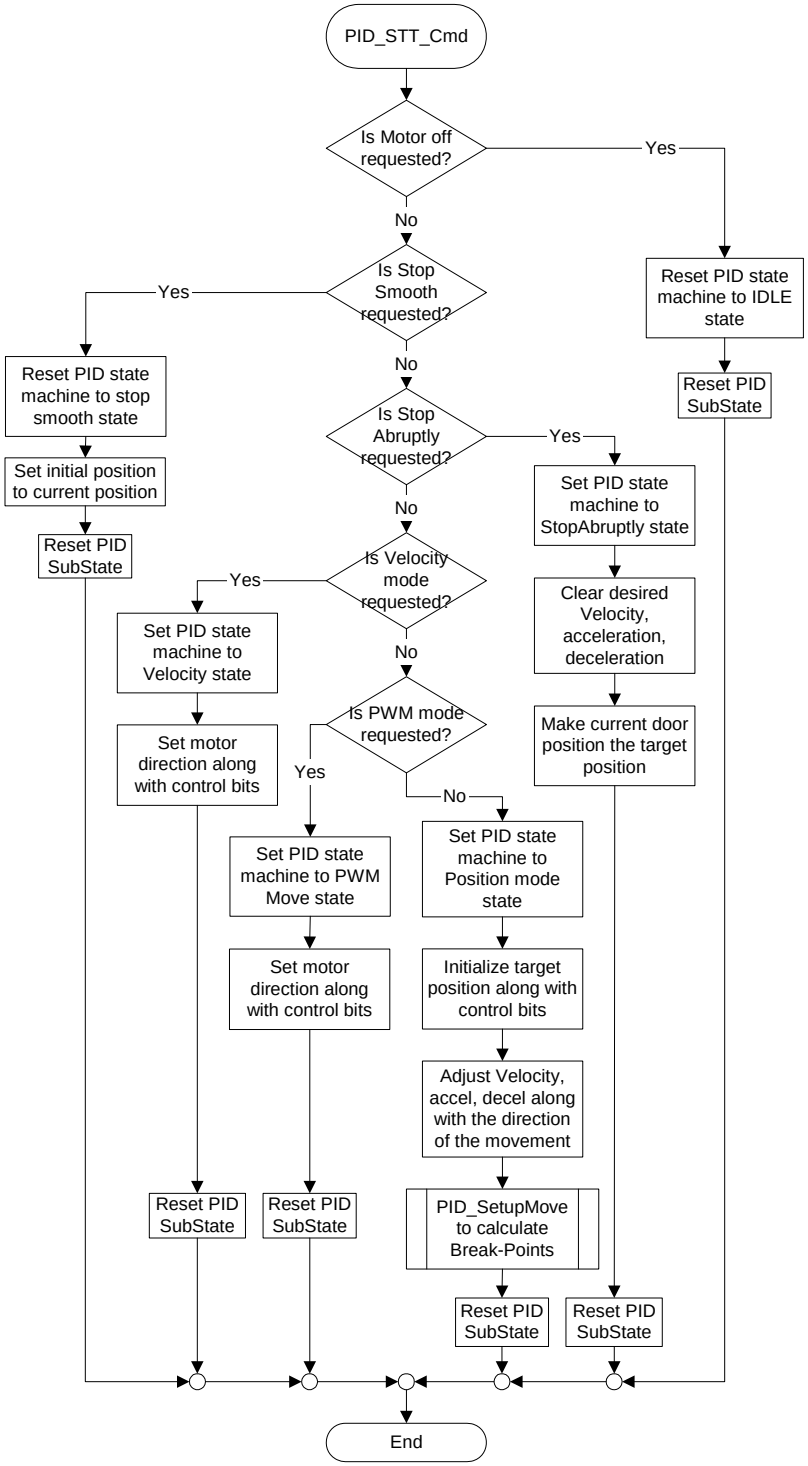

Figure 3-16: PID_STT_Cmd Flowchart

3.4.1.16 PID_UDF Function

Set flag into Signal register to notify the PID task has to update the filters on the next PID task execution. The flag bExecutingUDF is set into this function and reset at the next execution of the PID Task.

3.4.1.17 pid_Setup_Move Function

This function prepares the door moving when an STT command has been executed.
This function is called by the PID_STT_Cmd function.
The pid_Setup_Move function is used to generate the data needed for the Pid_Profile_Move_State function to identify points where the door shall have acceleration completed, when door shall starts deceleration and when door shall stops.
The pid_Setup_Move function calculates first the delta position of the requested move and extracts the direction at same time. The pid_Setup_Move can start a move with an initial velocity not null. It verifies the initial velocity to determine if it is higher than nominal velocity.
The calculation of profile type is done in order to verify if the requested movement is trapezoidal or triangle.
Formula to calculate the Vmax when Accel = Decel,

Vmax = dir * (sqrt ((Final Position – Initial Position) * Accel * dir))

or when Accel <>Decel.

$$Vmax = \frac{dir * (sqrt\ (((Initial\ Position – Final\ Position) * Accel * Decel * 2) – (Decel * Vo^2\ ))\ )}{(Accel – Decel)}$$

The trajectory is found to be triangle whenever the value of the difference between max allowed velocity and Vmax multiply by the direction (positive if opening and negative if closing) is lower than zero.
In case the trajectory is found to be triangle, the new calculated Vmax is used as absolute maximum velocity.

The number of acceleration samples is then calculated with the new Velocity. Sample_Accel = (Vmax – Vinitial) / Accel. This found number is then rounded at the highest integer whenever there is a reminder. The new accel value is then calculated with the new number of samples found.

NewAccel = (Vmax – Vinitial)/Sample_Accel (rounded up)

The number of samples for the deceleration time is calculated in the same manner to obtain a number rounded up.
The new deceleration is then calculated to match the new number of decel sample.
The number of samples for the constant speed section (in case the profile is not triangle) is then calculated with the new acceleration, deceleration. See Table 3-6 for detailed implementations of the equations.

Table 3-6: Equation for Final Accel, Decel and Speed Calculation

```
// Set the deceleration value
dNewDecelAdj = TrajectoryCurrent.dDecel;

// Calculate new Ta rounded-up
lTa = labs((long)(ceil((dNewVmaxAdj - dVo)/ dNewAccelAdj )));

// Calculate new Td rounded-up
lTd = ceil(-dNewVmaxAdj / dNewDecelAdj );

// Calculate new Tm rounded-down
dTemp1 = (dDeltaPosition - ((dVo + dNewVmaxAdj) * (double)lTa * 0.5) - (dNewVmaxAdj * (double)lTd
*0.5)) / dNewVmaxAdj;
lTm = labs((long)(floor(dTemp1)));

// Recalculate final acceleration
dAccel_Final = (dDeltaPosition - dVo *((lTa + lTm) + (double)(lTd)*0.5))/ ((double)lTa *
(((double)(lTa+ lTd)*0.5) + lTm));

// Recalculate final deceleration
dDecel_Final = -((dAccel_Final * (double)lTa + dVo)/ ((double)lTd));
```

Once the final acceleration, deceleration and velocity are calculated, the function stores the information into the profile structure. Table 3-3 shows the structure used to store the profile information. The current step, uiStepExec is reset to zero by the pid_Setup_Move function after calculation of the new parameters. The structure can hold three movements segments and one end state.
The Acceleration and deceleration phases are stored respectively in the record #0 and record #2. The numbers of samples of each segment are stored into the ulSample field. The constant speed segment is recorded in the structure by setting the dAccel filed of the record #1 to zero and write the number of samples into ulSample filed that the PID module shall keep the door into this segment.

3.4.1.18 pid_Velocity_Move_State Function

This function is the core of the PID module control when the door is controlled in velocity mode.
The PID_CommandTask call this function every 5 ms.

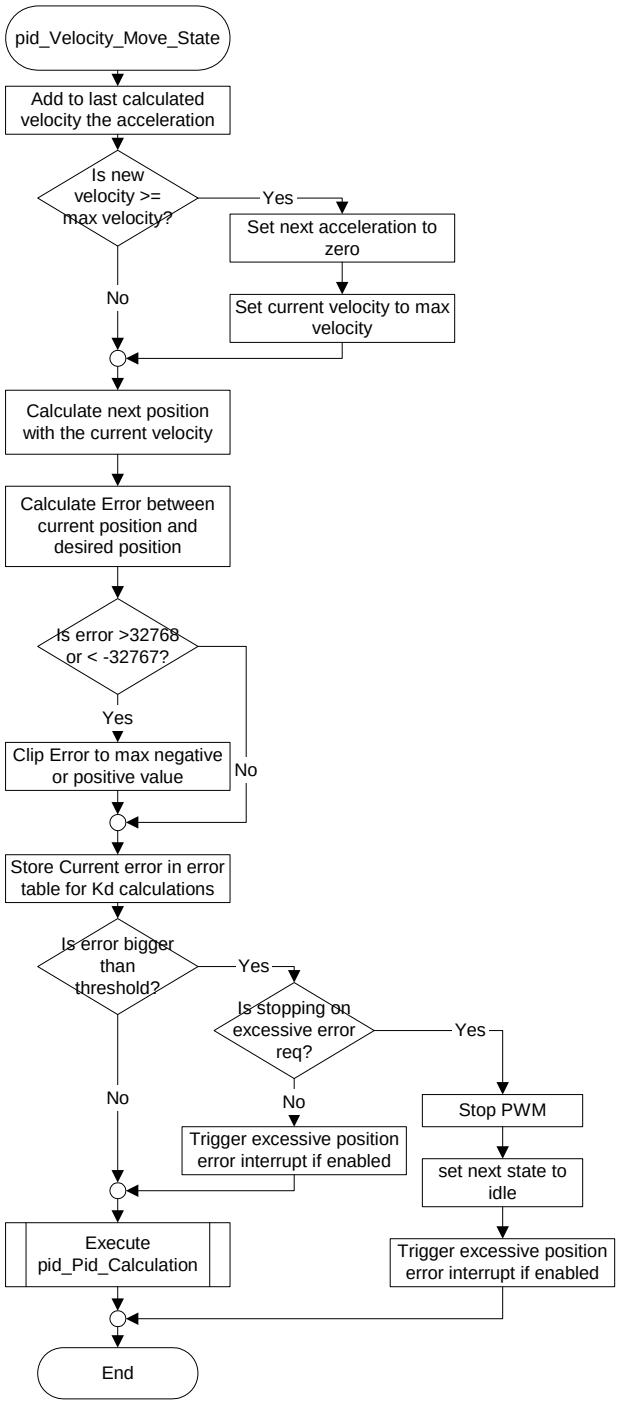

Figure 3-17: pid_Velocity_Move_State Flowchart

Door Obstruction Detection:
The obstruction sensitivity is based both on the position error and the derivative of the error. The error has to be higher than the limit and increase at a high rate (high derivative) to trigger an obstruction. If the rate is low, the obstruction is triggered only when twice the position error is reached.

3.4.1.19 pid_PWM_Move_State Function

This function is used by the PID module when the door is controlled in PWM mode. It starts in the Sub-State 1. The PID_CommandTask calls this function every 5 ms.

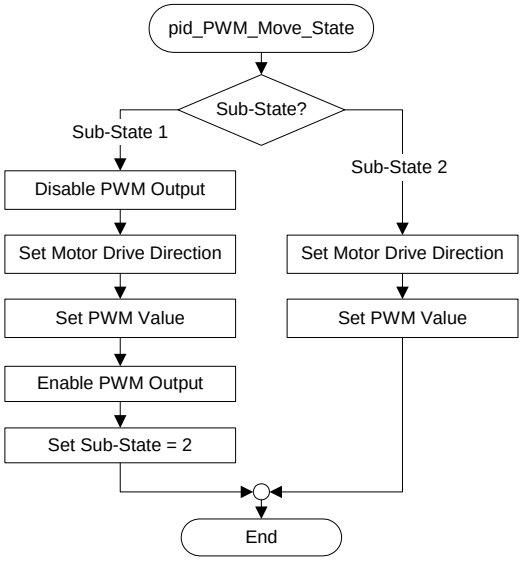

Figure 3-18: pid_PWM_Move_State Flowchart

3.4.1.20 pid_Profile_Move_State Function

This function is the core of the PID module control when the door is controlled in position mode. The PID_CommandTask calls this function every 5 ms. The movement is prepared prior to its execution upon execution of the STT Command. Segments of the movement are stored into the move structure, see Table 3-3 for content of the structure, and used by the Pid_Profile_Move_State. The Figure 3-19 shows the processing of the Pid_Profile_Move_State.

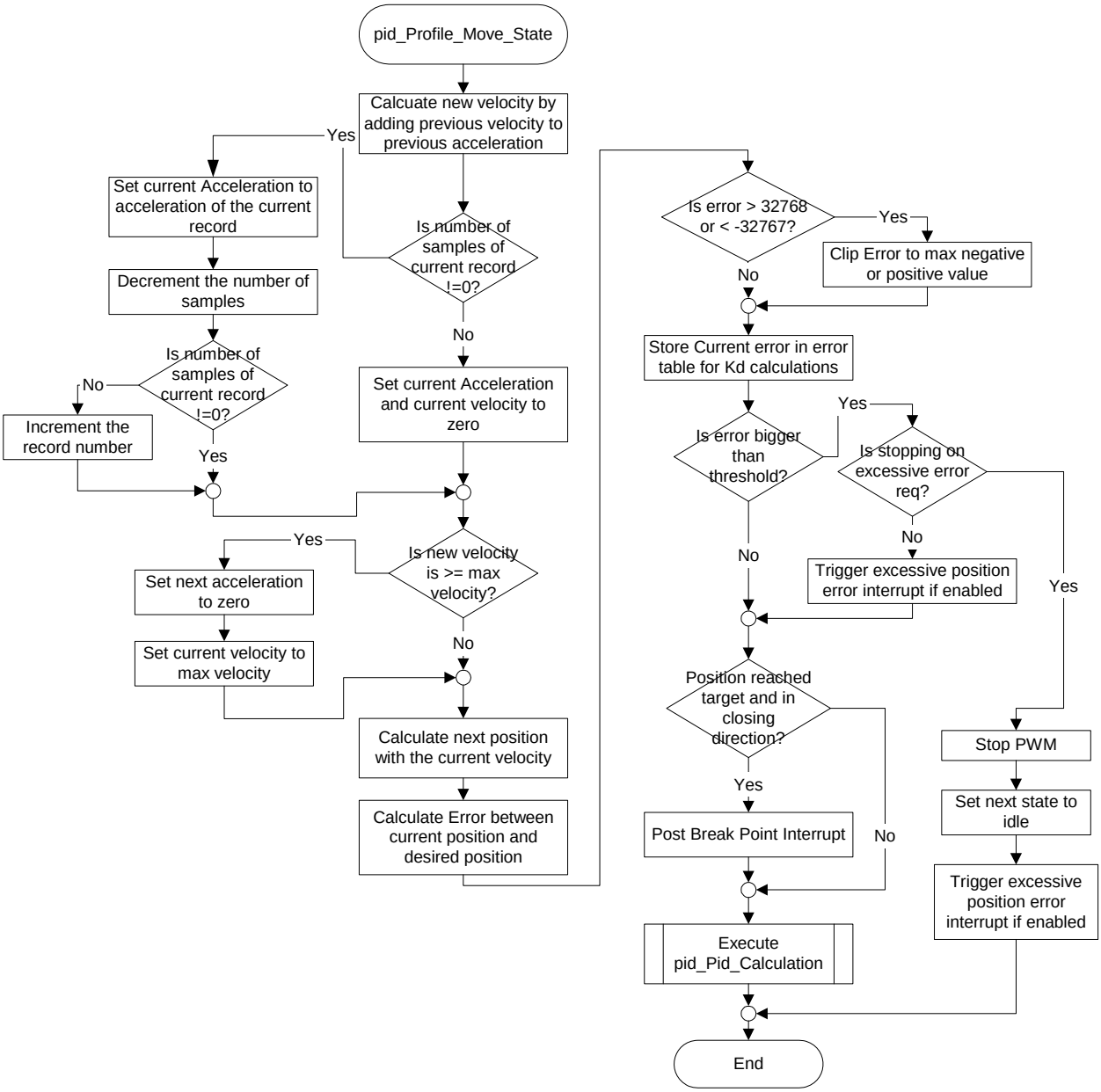

Figure 3-19: pid_Profile_Move_State Flowchart

Door Obstruction Detection:

The obstruction sensitivity is based both on the position error and the derivative of the error. The error has to be higher than the limit and increase at a high rate (high derivative) to trigger an obstruction. If the rate is low, the obstruction is triggered only when twice the position error is reached.

3.4.1.21 pid_Stop_Smooth_State Function

This function is the core of the PID module control when the door is in stopping mode. This state is used to stop the door before reaching the target position (or for stopping the door when in velocity mode) to decelerate the door as programmed in the deceleration value. This function is called by the PID_CommandTask every 5 ms. Figure 3-20 shows the pid_Stop_Smooth_State process.

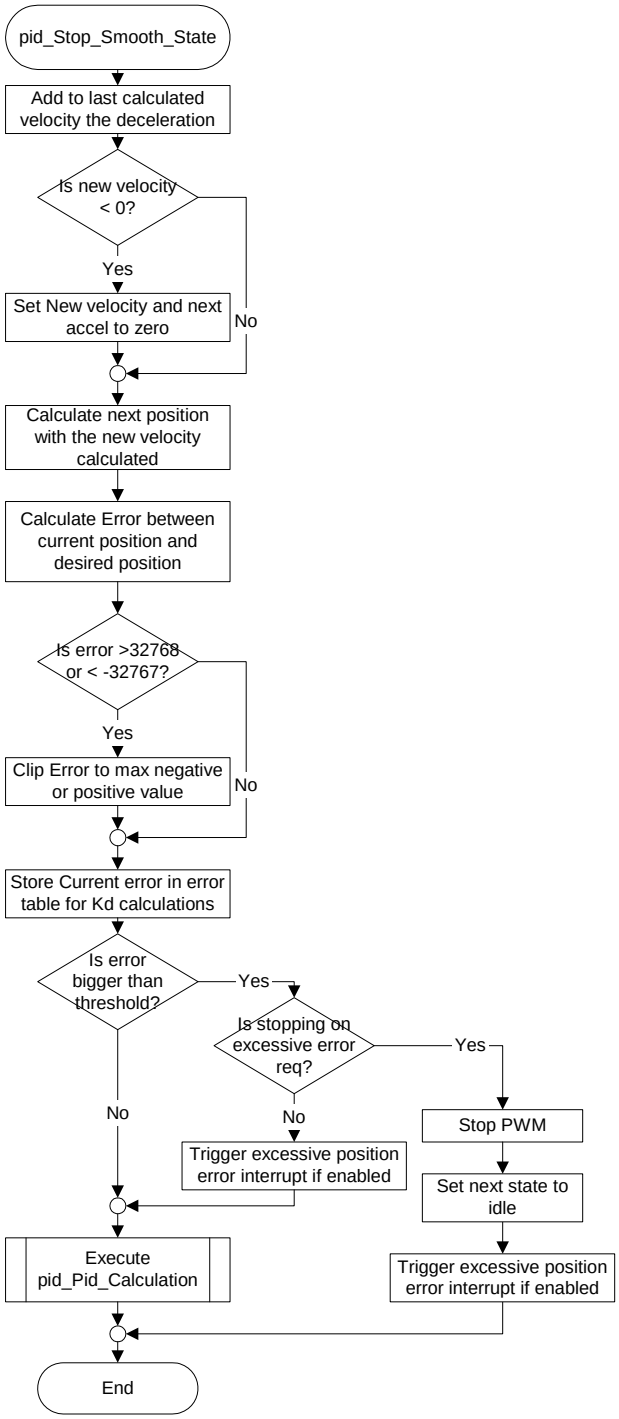

Figure 3-20: pid_Stop_Smooth_State Flowchart

3.4.1.22 pid_Stop_Abrupt_State Function

This function is the core of the PID module control when the door is needed to be stopped and to hold the current position. This function is called by the PID_CommandTask every 5 ms and is described in Figure 3-21.

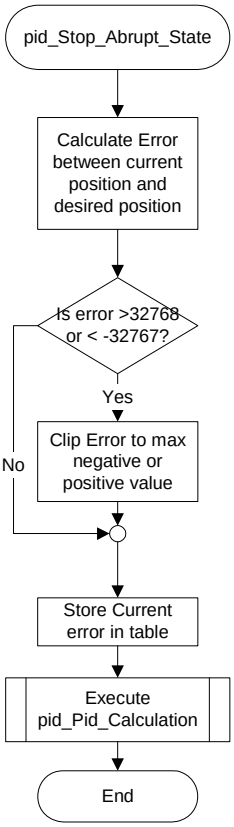

Figure 3-21: pid_Stop_Abrupt_State Flowchart

3.4.1.23 pid_Idle_State Function

This function is used whenever no movements of the door are requested and motor is not activated. The PID_CommandTask call this function every 5 ms. The processing of the function is to set the PWM output to zero.

3.4.1.24 pid_Pid_Calculation Function

The Pid_Pid_Calculation function is used to calculate the next command for the motor drive. The Pid_Pid_Calculation function uses the FilterCurrent structure that holds the current Kp, Kd, Ki, Il, and Si.
The proportional term is calculated by multiplying the current error with the Kp coefficient.
The derivative term is calculated by multiplication of the Kd coefficient and the difference between $Error_n$ and $Error_{n-1}$.
The integral term is calculated by adding the current error to the error sum register. The error sum register is then compared to the integral limit (Il) parameter and clipped to that value. The Ki coefficient is then multiplied by the new integral sum clipped.
To form the final command, all three results are summed and divided by 1024 to be scaled down. The polarity of the result is then extracted and the command is clipped to the maximum reload value for the PWM.

3.4.1.25 pid_Encoder_Test Function

The Encoder Test function verifies the encoder count error when the index interrupt is received. It sums the errors through all the door movement in one direction. The function verifies if there are 2000 encoder pulses since the last index in the current direction. The difference between the real encoder pulses value and the theoretical value to a global variable.
This helps to reduce timing issue due to interrupt latency (if one interrupt is delayed and gives a +3 encoder error, if the next interrupt is on time, it will give a –3 encoder error, the sum is 0 meaning no encoder pulse is lost). The error sum is reset when the direction changes.

3.4.1.26 PID_CommandTask Function

This function is the core of the PID module control. Figure 3-22 shows the processing of the PID_CommandTask.

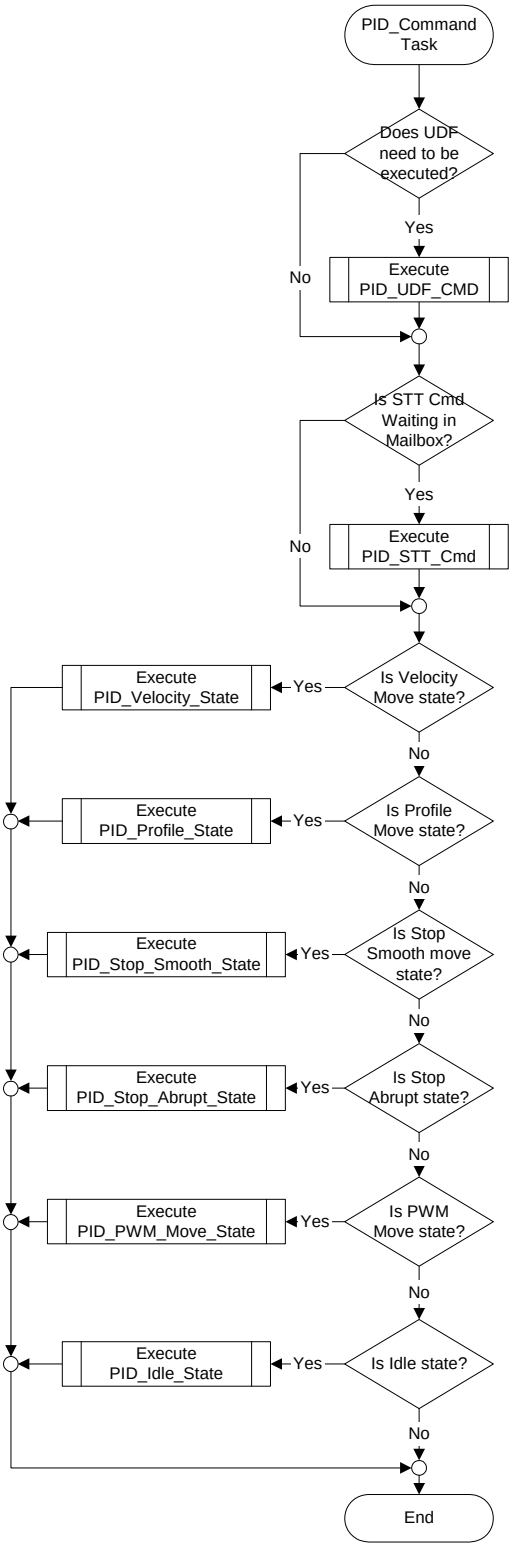

Figure 3-22: PID_CommandTask Flowchart

3.4.2 Data Entities Detailed Design

Not applicable.